

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (5, 5), 0)

def run_hough_lines(canny_low, canny_high, threshold, min_len):
    edges = cv2.Canny(gray, canny_low, canny_high)
    lines = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180,
                           threshold=threshold,
                           minLineLength=min_len, maxLineGap=0)

    out = img.copy()
    count = 0
    if lines is not None: # Check if lines were detected
        count = len(lines)
        # Reshape lines to (N, 4) to handle potential v
        # where lines can be (N, 1, 4) or sometimes (N, 2, 4)
        lines = lines.reshape(-1, 4)
        for l_coords in lines:
            # Unpack the 4 coordinates directly from 'l'
            x1, y1, x2, y2 = l_coords
            cv2.line(out, (x1, y1), (x2, y2), (0, 255, 0), 2)
    return edges, out, count

# Parameter sets to compare
settings = [
    {"canny_low": 50, "canny_high": 150, "threshold": 100},
    {"canny_low": 30, "canny_high": 100, "threshold": 100},
    {"canny_low": 80, "canny_high": 200, "threshold": 100}
]

results = []
for i, s in enumerate(settings):
    edges, out, count = run_hough_lines(**s)
    results.append((s, count))

```

```

cv2.imwrite(f"lines_set{i+1}.png", out)
cv2.imwrite(f"edges_set{i+1}.png", edges)

print(f"--- Settings Set {i+1} ---")
print(s, "-> lines detected:", count)

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(edges, cmap='gray')
plt.title(f'Edges (Set {i+1})')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(out, cv2.COLOR_BGR2RGB)) #
plt.title(f'Detected Lines (Set {i+1})')
plt.axis('off')
plt.show()

print("\n--- Circle Detection ---")
# Circle detection with varying param2 (sensitivity)
for p2 in [20, 30, 40]:
    circles_img = img.copy()
    circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT,
                               param1=100, param2=p2,

    n = 0
    if circles is not None:
        circles = np.uint16(np.around(circles))
        n = circles.shape[1]
        for i_circle in circles[0, :]: # Renamed loop v
            # draw the outer circle
            cv2.circle(circles_img, (i_circle[0], i_cir
            # draw the center of the circle
            cv2.circle(circles_img, (i_circle[0], i_cir

    print(f"param2={p2} -> circles detected: {n}")

plt.figure(figsize=(6, 6))
plt.imshow(cv2.cvtColor(circles_img, cv2.COLOR_BGR2

```

```
plt.title(f'Detected Circles (param2={p2})')  
plt.axis('off')  
plt.show()
```


